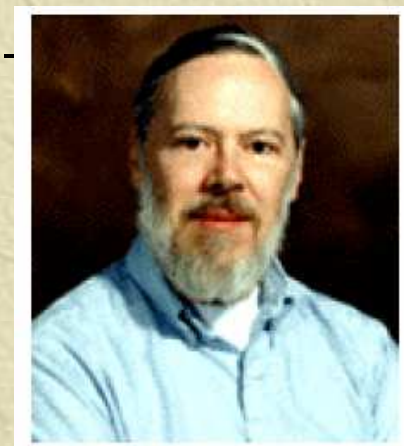
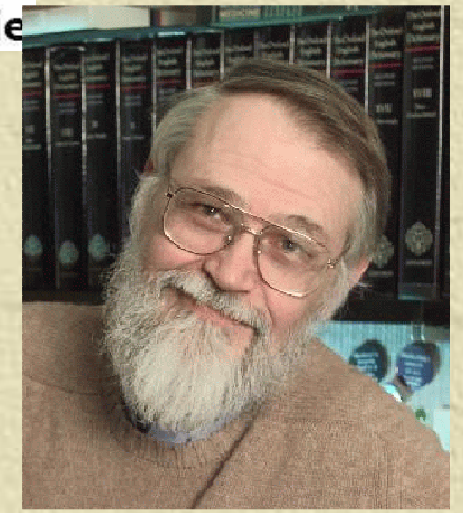
3. Introducere în limbajul C

Apariţia limbajului C



Dennis M. Ritchie



- Util pentru a scrie compilatoare, editoare, sisteme de operare, aplicaţii
- A fost dezvoltat şi implementat pe sistemul de operare UNIX pentru DEC (Digital Equipment Corporation) PDP11 (1972)

Limbajul B (Ken Thomson) – BCPL (Martin Richards)

1978 Brian Kernighan &Dennis M. Ritchie: "The C Programming Language" (ed. 2:1989)

Limbajul de programare C

- ~ 5000 limbaje de programare
- Unix (prima versiune) – pentru PDP 11 în limbaj de asamblare
- C – pentru Unix –> pentru sisteme de operare
- OS/2 (IBM) – asamblare -> C
- Pentru a scrie programe care fac apel la serviciile SO se folosesc uşor rutinele scrise în C (citirea unui fişier, crearea unui proces, etc.)
  - API (Application Programmers Interfaces) - pentru Unix, Windows NT, OS/2 definesc o interfaţă în C către SO.
- Număr mare de biblioteci scrise în C.

PC_1.C

Kernighan despre limbajul C

Un interviu cu Brian Kernighan, iulie 2000 - Mihai Budiu
mihaib+@cs.cmu.edu; http://www.cs.cmu.edu/~mihaib/

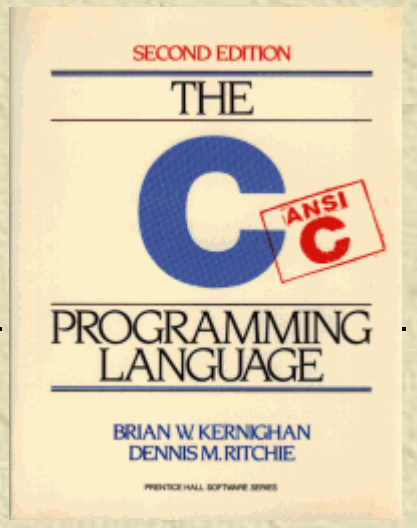
- *C are cel mai bun echilibru pe care l-am văzut vreodată între putere şi expresivitate.*
- *Poţi face aproape orice vrei programând într-un fel destul de evident şi ai întotdeauna un model mental bun a ceea ce se va întâmpla pe calculator; poţi prezice destul bine cât de repede o să meargă, înţelegi ce se petrece şi îţi dă libertate totală să faci ce vrei. … dar în ceea ce priveşte realizarea a ceva fără prea mult efort, nu am văzut nimic mai bun până în ziua de azi.*
- *Există alte limbaje reuşite pentru aplicaţii specifice, dar dacă ar fi să eşuez pe o insulă pustie cu un singur compilator, aş vrea un compilator de C.*

Kernighan despre limbajul C

Un interviu cu Brian Kernighan, iulie 2000 - Mihai Budiu
mihaib+@cs.cmu.edu; http://www.cs.cmu.edu/~mihaib/

- *Nu pot comenta despre trăsăturile negative; îţi reamintesc că C este în întregime creaţia lui Dennis Ritchie. Eu am contribuit doar la popularizarea sa, şi în particular nu pot spune ce e greu şi ce e uşor de compilat în C.*
- *Sunt câteva lucruri greşite în C: instrucţiunea switch putea fi mai bine proiectată, precedenţa unora dintre operatori este greşită, dar astea sunt lucruri mărunte şi lumea a învăţat să le evite.*
- *Cred că adevărata problemă a limbajului C este că nu-ţi pune la dispoziţie destule mecanisme pentru a scrie programe cu adevărat mari, pentru a crea ziduri de protecţie în interiorul programelor, care izolează feluritele bucăţi. Nu se pune problema că nu poţi face astfel de lucruri, că nu poţi simula programare orientată pe obiecte sau alte metodologii de programare în C, poţi să faci toate astea, dar compilatorul şi limbajul nu te vor ajuta.*
- *Dar dacă luăm în considerare că C are aproape 30 de ani şi că a fost creat când calculatoarele erau minuscule în comparaţie cu ceea ce avem acum, cred că e o creaţie admirabilă, care a trecut de testul scurgerii timpului extrem de bine. Nu sunt multe lucruri pe care le-aş schimba în C.*

Standardul ANSI C



- ✓ American National Standard Institute: ANSI C standard (1983 - 1989) – numărul documentului: #X3J11/99-090
- ✓ Update: 1999
- ✓ Defineşte: arhitectura, sintaxa, funcţiile bibliotecii C standard.
- ✓ Este standardul internațional pentru C

ANSI C - portabilitate

3.1. Mediul de programare Turbo C

- ✓ Un mediu de programare pune la dispoziţia programatorului facilităţi de scriere şi depanare a unui program într-un anumit limbaj.
- ✓ Mediul de programare Turbo C se bazează pe limbajul C.
- ✓ Acest mediu are un compilator care "traduce" programul scris în C într-un executabil.
- ✓ IDE – Integrated Development Environment

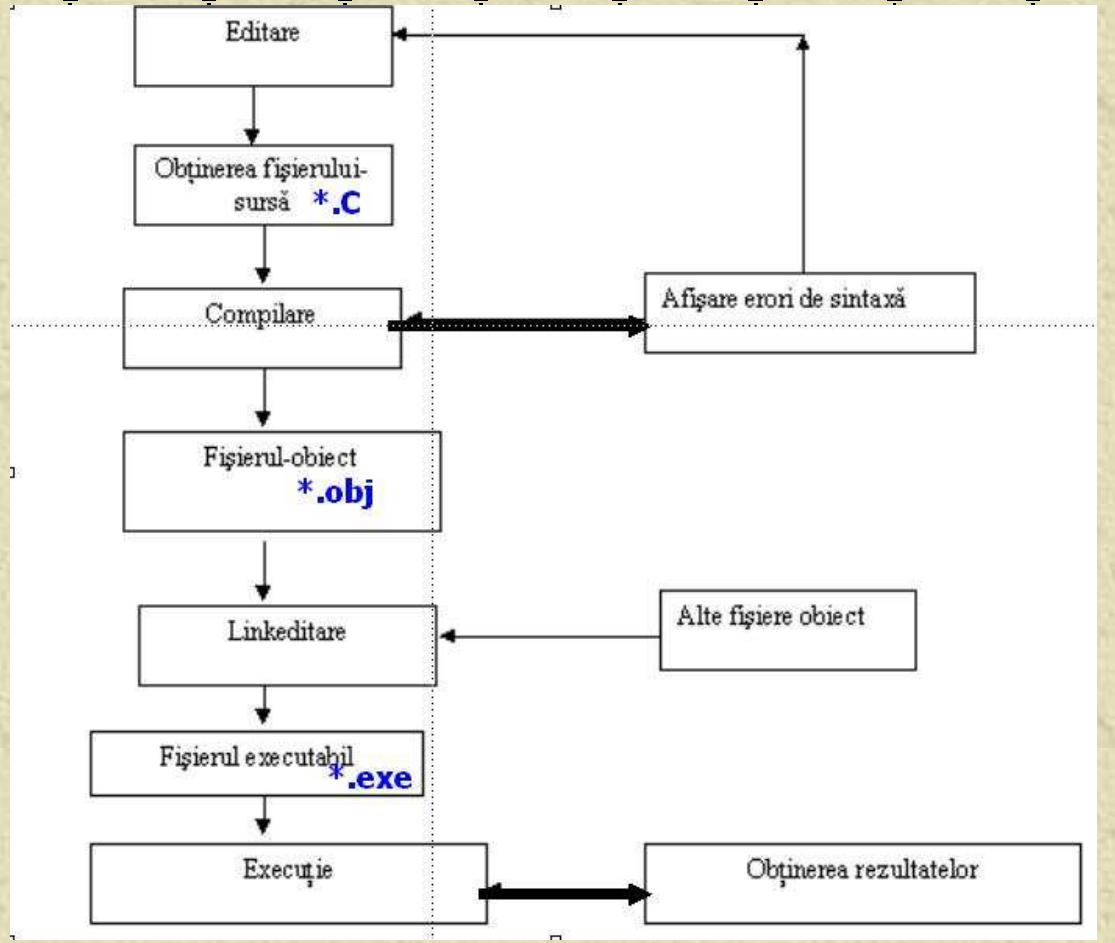
Ecranul conţine trei zone:

- ✓ Meniul principal
- ✓ Fereastra de editare
- ✓ Descrierea tastelor funcţionale

Mediul de programare TC pune la dispoziţia programatorului :

- ✓ Un editor de texte
- ✓ Un compilator
- ✓ Un link-editor
- ✓ Un sistem de "help"
- ✓ Ferestre multiple
- ✓ Posibilitatea de depanare a programului (Debug)

Crearea unui program implică următoarele etape clasice



Compilatoare C pentru diverse platforme

- ✓ Mainframes
- ✓ Personal computers (Borland C++, Microsoft Visual C++)
- ✓ Microcontrolere
- ✓ Procesoare numerice de semnal (Digital Signal Processors:DSP).

Compilare – Asamblare – Link-editare

	Extensie fişier intrare	Extensie fişier ieşire
Compilator	.C	.asm
Asamblor	.asm	.obj, .lib
Link-editor	.obj .lib	.exe

3.2. Structura unui program în TC

Un program în TC este structurat în general în trei părţi:

- ✓Secţiunea "#"
- ✓Declaraţii
- ✓Instrucţiuni

Structura unui program

(ANSI C)

- ✓ Declaraţii de funcţii
- ✓ Declaraţii de variabile externe

- ✓ Apelul funcţiei *main()* {
  - ✓ Declaraţii de variabile locale
  - ✓ Instrucţiuni }

- ✓ Definiţii de funcţii {
  - ✓ Declaraţii de variabile locale
  - ✓ Instrucţiuni
  - ✓ *return* valoare }

Pe parcurs - Comentarii /*...*/

Readability

Cât de uşor poate fi citit programul editat în C

Stil de programare

Exemplu:

-toate numele funcţiilor utilizator să înceapă cu literă mare (pentru a le desoebi de cele standard) şi să nu conţină caracterul de subliniere

-toate variabilele să conţină doar lietere mici

-variabilele să aibă o anumită semnificaţie (i,j,k,l – folosite pentru iteraţii)

-declaraţiile să preceadă funcţiile, funcţiile să preceadă funcţia *main*

-să nu fie folosită instrucţiunea *goto*

-…

```
 /*      Here is a comment.
  *      This is for demo.
  */
enum day { SUN=1, MON, TUE, WED, THU, FRI, SAT }
struct date {
  int            dd;     /* day no. 1-31 */
  enum day       dname;  /* weekday name */
  long           yyyy;   /* year */
};
/*       Another comment.
 *       Purpose of the function.
 */
  int
foo ( foo1_t const *f1, foo2_t *f2 )
{
      for (...) {
                 while (...
                        ...
                        if (error)
                               goto error;
                 }
          }
          ...
```

Stil de programare

- Recomandări
  - scrieţi cod **eficient**: cod scurt şi execuţie rapidă;
  - folosiţi reguli în aşa fel încât codul să fie **uşor de citit;**
  - codul să folosească pe cât posibil **toate facilităţile** bune ale limbajului.

Stil de programare

- La începutul programului scrieţi câteva informaţii despre condiţiile de realizare ale acestuia:
  - Numele programului
  - Ce realizează programul
  - Autorul programului
  - Copyright
  - Condiţii de utilizare (limite, capcane)
  - Orice altă informaţie folositoare

```
/*********************************************************
 * Analyze -- analyze the complexity of a program       *
 *                                                      *
 * Purpose:                                             *
 *      This program produces a set of statistics that  *
 *      are supposed to tell you how complex a program  *
 *      is.                                             *
 *                                                      *
 * Author: John Jones.                                  *
 *                                                      *
 * Copyright 1999 by John Jones.                        *
 *                                                      *
 * Warning: Compiling with optimization causes          *
 *      incorrect code to be generated.                 *
 *                                                      *
 * Restrictions: Works only on error-free C files.      *
 *      Does not know about pre-processor directives.   *
 *                                                      *
 * Algorithms: Uses a classic unbalanced binary tree    *
 *      for a symbol table.                             *
 *                                                      *
 * References: "Software complexity measurements",      *
 *      Flying Fingers Newsletter, May 5, 1995.         *
 *                                                      *
 * Output file format for raw data file:                *
 *              <magic number> (AC_DATA_MAGIC)          *
 *              <# statistics that follow>              *
 *                 <stat table index>                   *
 *                 <value>                              *
 *                 (Repeat for each stat.)              *
 *                                                      *
```

Stil de programare

```
/*******************************
* Textul care descrie programul        *
* Pe mai multe rinduri                 *
*******************************/
```

- Folosiţi comentarii – pentru a descrie o variabilă, o linie mai importantă, detalii de programare, capcane posibile; scrieţi comentariile în timpul editării programului.
- Nu creaţi fişiere foarte mari care sunt incomod de editat şi tipărit.
  - creaţi fişiere de maxim 2000-3000 de linii
- Liniile unui program nu trebuie să depăşească 72 de caractere (limita ecranului este 80), dar se recomandă linii pe cât posibil de scurte
- Folosiţi doar cele 95 de caractere tipăribile din standardul ASCII.

Structura unui program în TC

Secţiunea "#" – conţine directive (*include*) şi macroninstrucţiuni (*define*) - încep cu '#'.

Efect *include*: tratează textul din fişierul specificat în #*include* ca şi când ar face parte din programul curent.

- Biblioteca C standard
- Program C -> module = funcţii
- Funcţiile din biblioteca standard - operaţii: intrare, ieşire, matematice, etc.
- C Preprocessor (în *C Reference Card* - ANSI)

Structura unui program în TC

Declaraţii - în această secţiune programatorul defineşte (**PC_2.C**) toate entităţile pe care le va folosi în program: constante, variabile, funcţii.

Toate aceste entităţi vor avea un nume. Aceste nume (simboluri) sunt indentificatorii.

Identificatorii

- ✓ Identificatorii trebuie să înceapă cu o literă (mare sau mică) sau caracterul de subliniere (*underscore*) şi conțin litere, cifre şi caracterul de subliniere.
- ✓ TC face diferențiere între litere mari şi litere mici (este "*case-sensitive*").
  - TC1, Tc1, tc1 corespund unor identificatori diferiți.
- ✓ ANSI C: lungimea unui identificator:
  - 31 pentru identificator intern
  - 6 pentru identificator extern

PC_3.C

Identificatorii

- Identificatorii în C pot fi:
  - cuvinte rezervate
  - cuvinte utilizator.

Cuvintele rezervate

- sunt cuvintele predefinite ale limbajului C. Acestea nu se declară în program, au o semnificație stabilită la construcția limbajului. Pot fi:
  - cuvinte cheie: părți din program, instrucțiuni sau părți de instrucțiuni (const, if, for).
  - cuvinte standard: desemnează constante, tipuri și funcții standard (int, double, printf, scanf, sin, cos, INT_MAX).

Cuvintele utilizator

- sunt cuvinte stabilite de către programator pentru a desemna entități ale programului său (constante, variabile, tipuri, funcții, etichete).
- sunt create prin specificarea într-o declarație (de tip, variabilă, constantă, funcție).
- exemple: matr, i, num12, NumPrim, etc.
- ANSI C permite ca identificatorii care încep cu
  - două caractere de subliniere sau
  - un caracter de subliniere urmat de literă mare
- să fie asociate doar pentru utilizarea de către
- compilator.

Variabile/constante

- ✓ Variabilele sunt cantități care își pot modifica valoarea în cursul execuției programului, în timp ce constantele rămân fixe.

Constantele pot fi

- numere întregi sau reale (constante numerice);
- şiruri de caractere (constante alfanumerice);
- constante simbolice (identificatori).

Constante numerice

- ✓ Numerele pot fi cu semn sau fără semn. Numerele întregi pot fi prezentate în baza 10 sau 16.
- ✓ Exemple: 0xd->13; 0xA5; 10*16+5=165; 0xFF->255
- ✓ Numerele reale se pot scrie
  - parte întreagă.parte fracționară
  - mantisăE{+/-}exponent.
- ✓ Exemple: 2,75 -> 2.75; $3*10^6$ -> 3e6;

  $-2*10^{-2}$ -> -2e-2.

PC_4.C

Constante alfanumerice

- În acest caz caracterele au asociate coduri.
- Acestea reprezintă poziția într-un tabel standard care include toate caracterele accesibile pentru un anumit calculator.
  - Pentru IBM PC se foloseşte ASCII (American Standard Code for Information Interchange) cu 256 de caractere, de la 0 la 255, reprezentate în memorie pe 1 octet.
- Exemple: ‘ ’-> 32 ; ‘*’ -> 42; ‘A’ -> 65; ‘a’ -> 97.
- Cifrele şi literele mari şi mici ocupă zone compacte crescător şi alfabetic. În C, şirurile de caractere reprezintă o succesiune de caractere între ghilimele.

Constante alfanumerice

- Numărul de caractere din şir se numeşte lungimea şirului. Dacă lungimea şirului este 0 şirul este vid.
- Nu toate caracterele ASCII au asociat un caracter care poate fi tipărit, acestea fiind caracterele de control (1 - 31): Line Feed, Carriage Return, Esc.

- **PC_5.C**

Instrucțiuni

- cu ajutorul instrucțiunilor programatorul descrie operațiile care acționează în momentul execuției programului asupra entităților declarate anterior.

Instrucțiuni

PC_6.C

C – limbaj în care se pot

- scrie/citi texte
- lua decizii
- repeta anumite acțiuni
- realiza diverse operații

Bază pentru alte limbaje şi exemplu de programare structurată